

CS471: Introduction to Artificial Intelligence

Assignment #5

Malcolm Zartman

10/30/2025

In recent years, distributed energy resources such as solar and wind power stations present havoc to the distribution portion of the grid. The intermittent fluctuations of power coming from the generation source creates power quality issues and instability challenges, which make the possibility of system failures more frequent and causes extensive damage to the power system infrastructure. To tackle this issue, optimizing wind turbine performance is essential, and the analysis of current wave data is crucial. Accurate identification of oscillatory signals within this data is challenging but vital, as these signals indicate operational concerns.

In this assignment, you will work with this wind farm dataset.

You can download the dataset from here: [Train](#) and [Test](#). The starter code for the assignment can be accessed from [here](#).

There are two classes, one being a normal class and the other being an oscillation class. The goal is to perform classification using any machine learning algorithm of your choice. You can use the Scikit-learn toolbox to develop your models.

1. Describe what is a binary classification and multi-class classification problem and specify if the current problem is a binary or a multi-class one. (1 point)
2. Perform pre-processing if necessary. (2 points)
3. Fit a machine learning algorithm of your choice on the training dataset. (1 point)
Explain the reason behind selection of machine learning algorithm (2 points)
4. Tune hyperparameters (if any) based on the performance on the validation set or using cross-validation. Plot the hyperparameter value on the x-axis and performance metric on the y-axis. (2 points)
5. Evaluate it on test data and generate a classification report (2 points)

Colab link:

https://colab.research.google.com/drive/1EdVdrcbkW4PYgAAUc2mAKKmA6V_M2-IV?usp=sharing

1. Describe what is a binary classification and multi-class classification problem and specify if the current problem is a binary or a multi-class one. (1 point)

A binary classification problem involves predicting one of only two distinct outcomes, it either is a given class or it is not. In contrast, a multi-class classification problem involves predicting one of N possible categories, making it generally more complex. Our current task is a binary classification problem because the goal is to determine if the measured signal is one of only two states: Normal or Oscillation.

2. Perform pre-processing if necessary. (2 points)

Since no traditional ML model can inherently process segmented sequential data (where each sample contains 200 ordered time steps), I have to reduce the dimensionality and encode temporal information by calculating descriptive features (such as Root Mean Square and standard deviation) for every single 200-point window, thereby transforming the data into something a traditional ML model can easily process.

```
def extract_features(X):  
    """  
    Extracts time-domain features from segmented time series data.  
    Features are crucial for distinguishing the low-amplitude 'normal'  
    from the high-amplitude 'oscillation' signal.  
    """  
    features = {  
        'mean': np.mean(X, axis=1),  
        'std': np.std(X, axis=1),  
        'max': np.max(X, axis=1),  
        'min': np.min(X, axis=1),  
        'ptp': np.ptp(X, axis=1), # Peak-to-Peak (max - min)  
        'rms': np.sqrt(np.mean(np.square(X), axis=1)), # Root Mean Square  
        'skew': skew(X, axis=1),  
        'kurtosis': kurtosis(X, axis=1)  
    }  
    return pd.DataFrame(features)  
  
# Feature Engineering  
X_train_features_df = extract_features(X_train)  
X_test_features_df = extract_features(X_test)
```

After that feature engineering step, I can now handle some preprocessing by standardizing the numerical data, and label encoding the class label so the model can handle it easier. I use the StandardScaler to normalize the range of the 8 statistical features, ensuring it fits only on the training data and then applied to the test data to prevent data leakage. These scaled NumPy arrays are immediately converted back into DataFrames for easier feature management. Finally, the categorical class labels are converted to a binary numeric format using label encoding (with 'oscillation' becoming 1 and 'normal' becoming 0), which is required for binary classification model training.

```
# Scale the features using the StandardScaler fitted ONLY on the training
data.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_features_df)
X_test_scaled = scaler.transform(X_test_features_df)

# Convert scaled arrays back to DataFrames for easier handling
(X_train_scaled_df, X_test_scaled_df)
X_train_scaled_df = pd.DataFrame(X_train_scaled,
columns=X_train_features_df.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled,
columns=X_test_features_df.columns)

# Map labels to numeric values (1 for oscillation, 0 for normal)
train_y_numeric = (train_y['label'] == 'oscillation').astype(int)
test_y_numeric = (test_y['label'] == 'oscillation').astype(int)
```

3. Fit a machine learning algorithm of your choice on the training dataset. (1 point)
Explain the reason behind selection of machine learning algorithm (2 points)

Here is the initialization of the Random Forest ensemble model, where I am setting the number of decision trees `n_estimators` to 100 and the `random_state` to 42 for reproducible results. Crucially, I set the `class_weight` to 'balanced'; this tells the model to automatically assign higher penalties for misclassifying the rare 'oscillation' samples, effectively neutralizing the data imbalance during the training fit process to ensure proper fault detection.

```
# Initialize Random Forest Classifier, explicitly handling the imbalance
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

# Train the model
rf_model.fit(X_train_scaled_df, train_y_numeric)
```

I choose to use the Random Forest ML algorithm as it offers a strong balance of high performance and robustness, especially well-suited for classification problems with non-linear relationships. By aggregating the results of multiple decision trees (an ensemble technique called bagging), Random Forest typically yields superior generalization results by significantly reducing the risk of overfitting compared to a single decision tree.

4. Tune hyperparameters (if any) based on the performance on the validation set or using cross-validation. Plot the hyperparameter value on the x-axis and performance metric on the y-axis. (2 points)

This code executes GridSearchCV to systematically test 24 combinations of four critical Random Forest hyperparameters (n_estimators, max_depth, etc.) and identify the best-performing set. It uses StratifiedKFold for cross-validation, which is crucial because it ensures each of the five test folds maintains the same small ratio of the minority 'oscillation' class, guaranteeing a reliable and consistent evaluation. The process uses the ROC-AUC metric to judge model performance, ultimately selecting the best model (best_rf) for final testing.

```
# Define the enhanced grid of hyperparameters
param_grid = {
    'n_estimators': [100, 300],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

# The base model for tuning (re-defined)
```

```

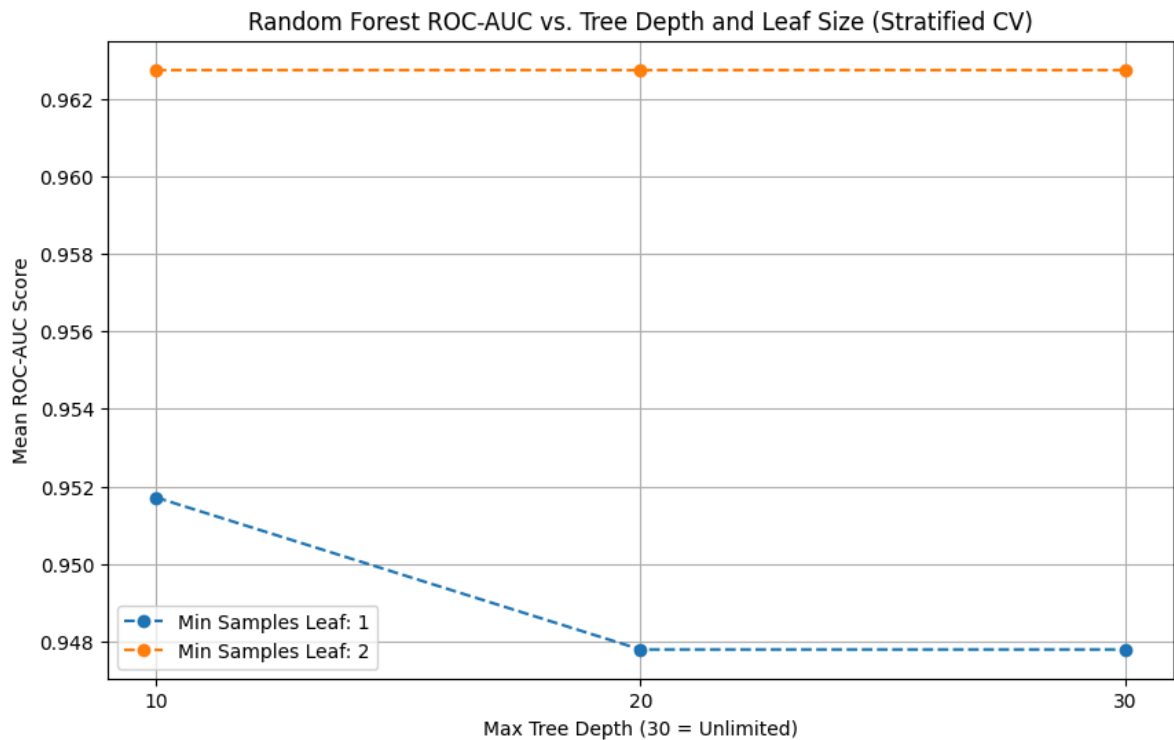
rf_tune = RandomForestClassifier(random_state=42,
                                class_weight='balanced')

# Stratified K-Fold CV
cv_stratified = StratifiedKFold(n_splits=5, shuffle=True,
                                random_state=42)
scorer = make_scorer(roc_auc_score)

grid_search = GridSearchCV(
    estimator=rf_tune,
    param_grid=param_grid,
    scoring=scorer,
    cv=cv_stratified,
    verbose=1,
    n_jobs=-1
)

print("\n--- Running GridSearchCV (Validation Phase) ---")
grid_search.fit(X_train_scaled_df, train_y_numeric)
best_rf = grid_search.best_estimator_

```



The visualization was chosen because the Max Tree Depth and Minimum Samples per Leaf are the two hyperparameters most critical for controlling model complexity and the bias-variance trade-off in a Random Forest. By plotting the Validation ROC-AUC against Max Depth, and using separate lines for the Leaf Size, we effectively displayed a multi-dimensional relationship in a single chart.

5. Evaluate it on test data and generate a classification report (2 points)

```
from sklearn.metrics import classification_report

# Test the model
y_pred = best_rf.predict(X_test_scaled_df)

# Evaluate the model
print("\n--- Model Evaluation ---")
print(f"Total Test Samples: {len(test_y_numeric)}")
print(f"Normal Samples: {test_y.value_counts().loc['normal'].item()}")
print(f"Oscillation Samples: {test_y.value_counts().loc['oscillation'].item()}")
print("\nClassification Report (Focus on F1-score and Recall for 'oscillation'):")
print(classification_report(test_y_numeric, y_pred,
target_names=['normal', 'oscillation']))
print(f"AUC-ROC Score: {roc_auc_score(test_y_numeric,
rf_model.predict_proba(X_test_scaled_df)[:, 1]):.4f}")
```



```
--- Model Evaluation ---
Total Test Samples: 957
Normal Samples: 837
Oscillation Samples: 120

Classification Report (Focus on F1-score and Recall for 'oscillation'):
      precision    recall  f1-score   support

   normal       0.97       0.97       0.97        837
  oscillation    0.80       0.82       0.81        120

   accuracy                0.95        957
  macro avg       0.89       0.90       0.89        957
 weighted avg       0.95       0.95       0.95        957

AUC-ROC Score: 0.9831
```

The final test results demonstrate excellent model performance with an AUC-ROC Score of 0.9831. For the minority 'oscillation' class, the model achieved a Recall of 0.82 and an F1-Score of 0.81. We specifically highlight these metrics because the data is highly unbalanced, and they confirm the model's strong ability to detect faults. We prioritize Recall over Precision in this specific fault detection scenario because the cost of a False Negative (missing an actual dangerous fault/oscillation) is typically much higher than the cost of a False Positive (a false alarm). Achieving a Recall of 82% means the model successfully detects 82% of all true 'oscillation' events, which is the primary goal of the system.